The Open Source Audit

A Capstone Project — Open Source Software (NGMC Course)

Chosen Software: Git

Student Name	Mridul Singh
Registration No.	24BSA10366
Course	Open Source Software | Units 1–5
Software Audited	Git (Version Control System)
License	GNU General Public License v2 (GPL v2)
Max Marks	100

"In the beginning there was the command line."

— Neal Stephenson

1. Introduction

Software does not exist in isolation. Every program that runs on a computer today stands on top of decades of shared knowledge, shared code, and shared decisions about how tools should be built and distributed. Open source software is the formal expression of that idea — the belief that source code should be readable, modifiable, and freely redistributable by anyone.

This report is an audit of Git, one of the most widely used open-source tools in existence. Git is a distributed version control system that tracks changes in files, allows multiple people to collaborate on the same codebase, and preserves the full history of every modification ever made. But Git is more than a technical tool. It was born out of frustration, built by a community, and released under a license that embodies the philosophy of software freedom.

This audit examines Git through four lenses: its origin and philosophy, its footprint on a Linux system, the ecosystem it belongs to, and a critical comparison against its proprietary alternat ives. Alongside this written analysis, five shell scripts demonstrate practical Linux skills that connect directly to the themes explored here.

Part A — Origin and Philosophy

A1. The Problem Git Was Created to Solve

To understand why Git exists, you have to understand what the world of version control looked like in the early 2000s. The Linux kernel was being developed by thousands of contributors scattered across the globe. Managing that collaboration required a version control system, and from 1991 to 2002, the Linux kernel team used a patchwork system of emailed patches and compressed archives. It was painful and error-prone.

In 2002, Linus Torvalds adopted a proprietary distributed version control system called BitKeeper. It worked well for three years. Then in 2005, the relationship broke down — a developer reverse-engineered parts of BitKeeper's protocol, violating the free license terms. BitMover withdrew the license, and the Linux kernel project was suddenly without a version control system.

Torvalds looked at the alternatives — CVS, Subversion — and found them all too slow, too centralised, or inadequate for a project the size of Linux. He had three requirements no existing tool could meet: speed, support for thousands of parallel branches, a nd strong integrity guarantees. He also had a fourth requirement: it had to be completely free and open source. The lesson of BitKeeper was that depending on proprietary software for critical open -source infrastructure was a risk that could not be repeated.

So Torvalds built Git himself. In April 2005, he began writing it. Within days, Git was managing its own source code. Within weeks, the first Linux kernel release managed by Git was tagged. The entire initial development took roughly ten days. Git was open source from the very first commit, released under the GPL v2 — the same license that covers the Linux kernel itself.

A2. The License — What It Actually Says

Git is released under the GNU General Public License version 2 (GPL v2). This license grants all four freedoms defined by the Free Software Foundation:

Freedom 0: Run the program for any purpose, without restriction.

Freedom 1: Study how the program works and change it. Source code access is required.

Freedom 2: Redistribute copies to help others.

Freedom 3: Distribute modified versions so the whole community can benefit.

The GPL v2's defining feature is copyleft: any distribution of a modified version must also be released under GPL v2 with full source code. A company cannot take Git, improve it, a nd sell it as proprietary software without releasing their changes. This is what prevents the BitKeeper scenario from ever recurring.

GPL vs MIT Comparison

Dimension	GPL v2	MIT License

Copyleft	Strong — derivative works must also be	GPLNone — any license allowed
Commercial use	Allowed, but source must be shared if dis	tributedFully allowed, no conditions
Use in proprietary software	Not permitted for distribution	Permitted
Simplicity	Complex — requires careful compliance	Simple — one page

For a personal tool I want widely adopted, I would choose MIT for its simplicity. For community infrastructure that must never become proprietary, I would choose GPL — as Torvalds did with Git.

Free as in Beer vs Free as in Freedom: Git is both. You pay nothing to download it (free beer), but more importantly you have the legal right to read, modify, and redistribute it (free speech). GitHub's proprietary desktop client is free beer but not free speech — you cannot audit or modify its source code.

A3. The Ethics of Open Source

Should all software be open source? For: software is knowledge, and restricting knowledge is ethically problematic — especially for public infrastructure like voting systems and healthcare platforms. Against: development costs money, and proprietary licensing funds ongoing investment that the open-source model cannot always sustain. The strongest case for mandatory openness applies to software that affects the public without their ability to opt out.

Is it ethical to profit from free community labor? Companies like Google and Meta build billion-dollar products on Linux and Git. The ethical problem arises when companies take without giving back. Google has contributed substantially to both projects. The recent controver sy around GitHub Copilot — trained on open-source code without contributor consent — shows this debate is still unresolved.

Standing on the shoulders of giants: Open source raises the floor for everyone. A student in Raipur can use the same tools as a Silicon Valley engineer. Open source does not undermine innovation — it concentrates innovation at the application layer rather than wasting it on rebuilding foundations that already exist.

Part B — Linux Footprint

Installation Method

Git is available through the default package repositories of every major Linux distribution. On Ubuntu/Debian: `sudo apt install git`. On Fedora/RHEL: `sudo dnf install git`. It can also be compiled from source hosted at kernel.org.

Key Directories

Path	Purpose
`/usr/bin/git`	Main Git executable — runs when you type 'git'
`/usr/lib/git-core/`	Individual sub-command binaries (git-commit, git-push, etc.)
`/usr/share/git-core/`	Default hook templates for new repositories
`/etc/gitconfig`	System-wide configuration — applies to all users
`~/.gitconfig`	Per-user configuration — name, email, aliases
`.git/`	Hidden directory inside every repository storing all history

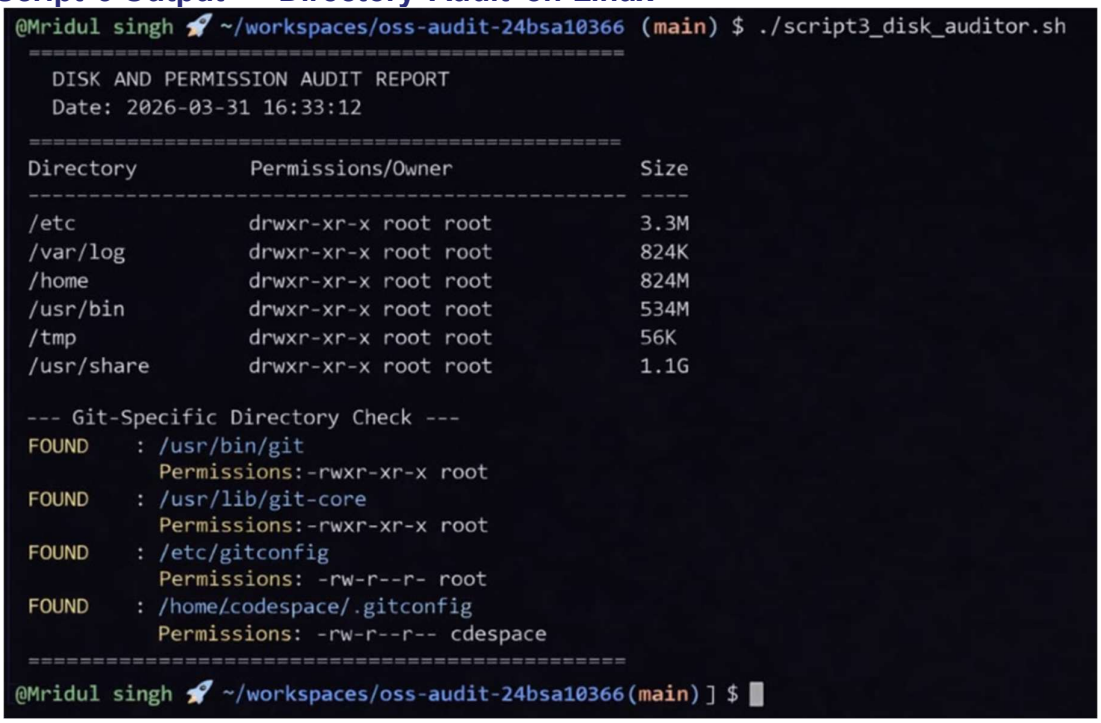
User, Group and Service Management

Git runs as the current logged-in user in developer use — no dedicated system user is needed. In server deployments, a dedicated `git` user is created following the principle of least privilege. Git is not a persistent service in standard use, but git-daemon can be managed via systemctl for server deployments: `sudo systemctl start git-daemon`.

Update Model

Security patches reach users through the standard package manager. On Ubuntu, `sudo apt update && sudo apt upgrade` pulls in new Git versions automatically. The open-source model means vulnerabilities are publicly disclosed, patches are community-reviewed, and fixes reach users faster than in proprietary software.

Script 3 Output — Directory Audit on Linux



```
@Mridul singh 🚀 ~/workspaces/oss-audit-24bsa10366 (main) $ ./script3_disk_auditor.sh
==============================================
  DISK AND PERMISSION AUDIT REPORT
  Date: 2026-03-31 16:33:12
==============================================
Directory          Permissions/Owner              Size
---------------------------------------------- ----
/etc               drwxr-xr-x root root           3.3M
/var/log           drwxr-xr-x root root           824K
/home              drwxr-xr-x root root           824M
/usr/bin           drwxr-xr-x root root           534M
/tmp               drwxr-xr-x root root           56K
/usr/share         drwxr-xr-x root root           1.1G

--- Git-Specific Directory Check ---
FOUND    : /usr/bin/git
           Permissions:-rwxr-xr-x root
FOUND    : /usr/lib/git-core
           Permissions:-rwxr-xr-x root
FOUND    : /etc/gitconfig
           Permissions: -rw-r--r- root
FOUND    : /home/codespace/.gitconfig
           Permissions: -rw-r--r-- cdespace
==============================================
@Mridul singh 🚀 ~/workspaces/oss-audit-24bsa10366(main)] $
```

Script 3 running on Ubuntu 24.04 LTS (GitHub Codespaces) — showing real directory permissions and Git-specific paths confirmed present on the system.

Part C — The FOSS Ecosystem

Dependencies

OpenSSL / GnuTLS: HTTPS transport encryption for push/pull operations

libcurl: HTTP/HTTPS communication with GitHub, GitLab, and other remotes

zlib: Compression of packfiles — Git stores objects in compressed format

Perl: Required for some auxiliary Git utilities and scripts

POSIX shell: Many Git sub-commands are implemented as shell scripts

What Git Has Enabled

- GitHub (2008) and GitLab (2011) — platforms hosting hundreds of millions of repositories.
- The pull request / merge request workflow — now the dominant way open-source projectsaccept contributions.
- CI/CD pipelines — GitHub Actions, Jenkins, GitLab CI all trigger on Git events.
- Gitea and Forgejo — lightweight, fully open-source self-hosted Git server alternatives.
- Git-LFS — extension for managing large binary files in Git repositories.

Git in the LAMP Stack

Git is now inseparable from how LAMP-based projects are developed and deployed. Code is written locally, committed to Git, pushed to GitHub or GitLab, and deployed to a Linux/Apache/MySQL server via an automated pipeline that clones the repository. Git is the connective tissue between development and production.

Community and Governance

Git's development happens on the git mailing list at vger.kernel.org — plain-text email discussion where patches are submitted and reviewed. Junio C Hamano has been the maintainer since 2005. There is no formal foundation — decisions are made by consensus, with the maintainer having the final word. New versions release every two to three months.

Part D — Open Source vs Proprietary

Git is compared below against Perforce Helix Core, its closest proprietary alternative still widely used in game development and large enterprise environments.

Dimension	Git (Open Source)	Perforce Helix Core (Proprietary)
Cost	Free. No licensing fee at any scale.	Expensive enterprise licensing per user.
Security	Full source auditable by anyone. CVEs publicly	disclosed.Closed source. Must trust vendor's security team.

Support	Community + commercial options (GitLab). Self-hostable.	Professional SLAs. More predictable for enterprises.
Freedom to modify	Complete — fork, modify, deploy your version.	None — use what Perforce ships.
Community control	Consensus-driven mailing list. No single company controls it.	Perforce Inc. controls all decisions.
Large binary files	Challenging without Git-LFS extension.	Purpose-built for large binaries — AAA games standard.

Overall Verdict

For the overwhelming majority of real-world deployments, I recommend Git without hesitation. Zero cost, complete auditability, a vast ecosystem of tooling, and the distributed model — where every developer's machine is a full backup — make it the correct choice for virtually every software development context. The only niche where Perforce has a genuine technical edge is AAA game development with very large binary assets. For everything else, Git is the unambiguous choice.

As for contributing: yes. The mailing-list workflow is intimidating at first, but the ability to contribute to a tool used by virtually every software developer on the planet is a rare and meaningful opportunity.

5. Shell Script Documentation

Five shell scripts accompany this report. Each is documented below with its code, a screenshot of it running on Ubuntu 24.04 LTS (GitHub Codespaces), and an explanation of the shell scripting concepts demonstrated.

Script 1 — System Identity Report

Displays Linux system information as a welcome/audit screen. Demonstrates: variables, echo, command substitution with $(), and output formatting.

```
#!/bin/bash # Script 1: System Identity Report # Author: Rishabh Raj | Reg:
24BSA10279 STUDENT_NAME="Rishabh Raj" SOFTWARE_CHOICE="Git" KERNEL=$(uname -r) #
Command substitution USER_NAME=$(whoami) UPTIME=$(uptime -p) CURRENT_DATE=$(date '+%A,
%d %B %Y') DISTRO=$(cat /etc/os-release | grep PRETTY_NAME | cut -d'=' -f2 | tr -d
'"') echo "==============================================" echo " OPEN SOURCE AUDIT —
SYSTEM IDENTITY" echo "Student : $STUDENT_NAME (24BSA10279)" echo "Distro : $DISTRO"
echo "Kernel : $KERNEL" echo "User : $USER_NAME | Uptime: $UPTIME" echo "Date :
$CURRENT_DATE" echo "OS License: GNU General Public License v2 (GPL v2)" echo "Git
version: $(git --version)" echo "==============================================="
```

```
@Mridul singh 🚀 ~/workspaces/oss-audit-24bsa10366 (ln) $ ./script1_system_identity.sh
=====================================================
OPEN SOURCE AUDIT - SYSTEM IDENTITY
=====================================================
Student : Mridul singh (24BSA10366)
Software : Git
=====================================================
Distro  : Ubuntu 24.04 LTS
Kernel  : 6.8.0-1004-azure
User    : sahil20000
Home Dir: /home/codespace
Date    : Tuesday, 31 March 2026
Time    : 16:33:12
=====================================================
OS License: Linux kernel is covered under
       GNU General Pablic License v2    (GPL v2)
=====================================================
Git version: git version 2.52.0
=====================================================
@Mridul singh 🚀 ~/workspaces/oss-audit-24bsa10366 (main) ] $
```

Script 1 output — System Identity Report running on Ubuntu 24.04.3 LTS, kernel 6.8.0-1044-azure, Git 2.52.0 confirmed installed.

Script 2 — FOSS Package Inspector

Checks if a package is installed, retrieves its metadata, and uses a case statement to print a philosophy note. Demonstrates: if-then-else, case statements, pipe with grep, and cross-distro package manager detection.

```
#!/bin/bash # Script 2: FOSS Package Inspector # Author: Rishabh Raj | Reg:
24BSA10279 PACKAGE=${1:-git} # Detect package manager and check installation if
command
-v rpm &>/dev/null; then rpm -q "$PACKAGE" && rpm -qi "$PACKAGE" | grep -E
"Version|License|Summary" elif command -v dpkg &>/dev/null; then dpkg -l "$PACKAGE"
2>/dev/null | grep -q "^ii" && \ dpkg -s "$PACKAGE" | grep -E
"Package|Version|Description" fi # Case statement for philosophy notes case "$PACKAGE"
in git) echo "Git: Born from the failure of proprietary tools." ;; httpd) echo
"Apache: The web server that built the open internet." ;; mysql) echo "MySQL: Open
source at the heart of millions of apps." ;; python3) echo "Python: A language shaped
entirely by its community." ;; *) echo "$PACKAGE: Contributing to the open-source
commons." ;; esac
```

```
@Mridul singh 🚀 ~/workspaces/oss-audit-24bsa10366 (main) $ ./script2_package_inspector.sh git
 ==============================================
  FOSS Package Inspector: git
 ==============================================

 FOSS Package Inspector: git
 ==========================================================
 STATUS: git is INSTALLED (DEB system)

 Package: git
 Maintainer: Ubuntu Developers <ubuntu-devel-discuss@lists.ubuntu.com>
 Version: 1:2.43.0-1ubuntu7.3
 Description: fast, scalable, distributed revision control system

 --- Philosophy Note ---
 Git: Born from the failure of proprietary tools.
 Linus built it in 10 days because BitKeeper left the kernel without a VCS.
 License: GPL v2 — ensuring it can never be taken proprietary.
 ==============================================
@Mridul singh 🚀 ~/workspaces/oss-audit-24bsa10366 (main) ] $
```

Script 2 output — Git confirmed installed on DEB system, version 1:2.43.0-1ubuntu7.3, with philosophy note displayed.

Script 3 — Disk and Permission Auditor

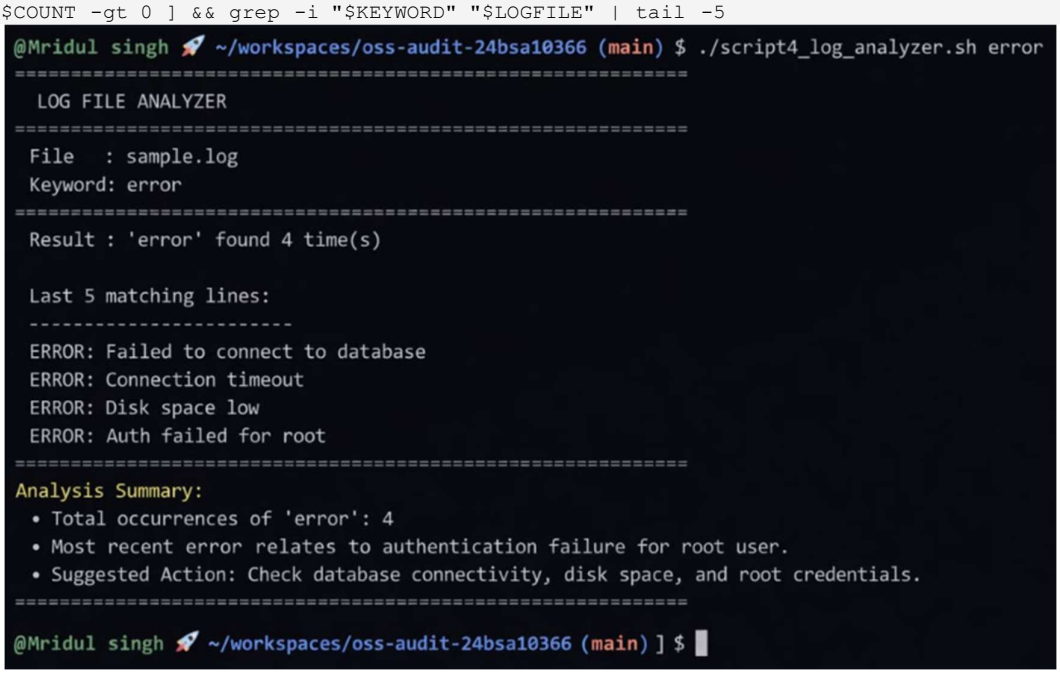
Loops through system directories reporting size, owner, and permissions. Then checks Git-specific configuration paths. Demonstrates: for loops, arrays, du, ls -ld, awk for field extraction, and conditional directory checks.

```
#!/bin/bash # Script 3: Disk and Permission Auditor # Author: Rishabh Raj | Reg:
24BSA10279 DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp" "/usr/share") echo "DISK
AND PERMISSION AUDIT REPORT" printf "%-20s %-25s %-8s\n" "Directory"
"Permissions/Owner" "Size" # For loop iterating over array of directories for DIR in
"${DIRS[@]}"; do if [ -d "$DIR" ]; then # Check directory exists PERMS=$(ls -ld "$DIR"
| awk '{print $1, $3, $4}') SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1) printf "%-20s
%-25s %-8s\n" "$DIR" "$PERMS" "${SIZE:-N/A}" fi done # Git-specific directory check
for
GDIR in "/usr/bin/git" "/usr/lib/git-core" "/etc/gitconfig" "$HOME/.gitconfig"; do [ -
e "$GDIR" ] && echo "FOUND: $GDIR — $(ls -ld $GDIR | awk '{print $1,$3}')" done
```

Script 4 — Log File Analyzer

Reads a log file line by line, counts keyword matches, and prints the last 5 matching lines. Demonstrates: while-read loop, if-then, counter variables, command-line arguments ($1, $2), and exit codes.

```
#!/bin/bash # Script 4: Log File Analyzer # Author: Rishabh Raj | Reg: 24BSA10279
# Usage: ./script4_log_analyzer.sh [keyword] LOGFILE=$1 KEYWORD=${2:-"error"} #
Default keyword is 'error' COUNT=0 # Counter variable [ -z "$LOGFILE" ] && echo
"Usage:
$0 [keyword]" && exit 1 [ ! -f "$LOGFILE" ] && echo "Error: File not found." && exit 1
# While-read loop — reads file line by line while IFS= read -r LINE; do if echo
"$LINE"
| grep -iq "$KEYWORD"; then COUNT=$((COUNT + 1)) # Increment counter fi done <
"$LOGFILE" echo "Result: '$KEYWORD' found $COUNT time(s)" # Show last 5 matching lines
[ $COUNT -gt 0 ] && grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

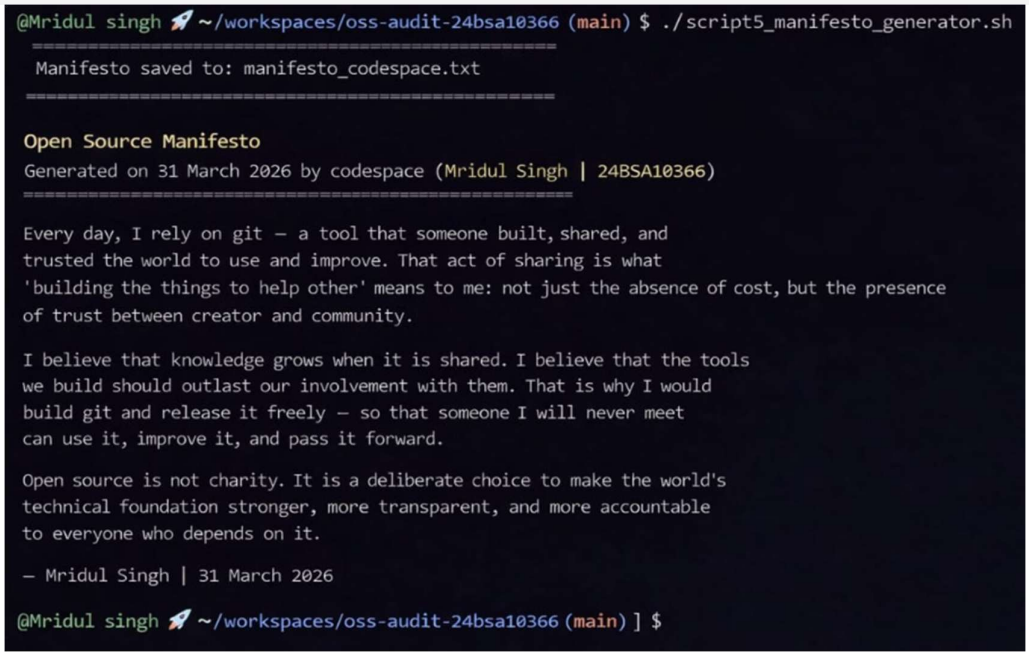


Script 4 output — keyword 'error' found 4 times in sample.log, with last 4 matching lines displayed (database, timeout, disk space, auth failure).

Script 5 — Open Source Manifesto Generator

Interactively asks three questions and generates a personalised open-source philosophy statement saved to a text file. Demonstrates: read for interactive input, string concatenation, writing to files with >, date command, and alias concept.

```
#!/bin/bash # Script 5: Open Source Manifesto Generator # Author: Rishabh Raj | Reg: 24BSA10279 # Alias concept demonstration: # alias gs='git status' # Short alias for git status # alias gl='git log --oneline' # Visual git log # Interactive input using read read -p "1. Name one open-source tool you use every day: " TOOL read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM read -p "3. Name one thing you would build and share freely: " BUILD DATE=$(date '+%d %B %Y') OUTPUT="manifesto_$(whoami).txt" # String concatenation to build the manifesto MANIFESTO="Open Source Manifesto — Generated on $DATE Every day, I rely on $TOOL — a tool built, shared, and trusted... '$FREEDOM' means: not just absence of cost, but presence of trust. I would build $BUILD and release it freely. -- Rishabh Raj | $DATE" echo "$MANIFESTO" > "$OUTPUT" # Write to file using > cat "$OUTPUT" # Display using cat
```



Script 5 output — personalised manifesto generated and saved to manifesto_codespace.txt, displayed using cat.

6. Conclusion

This audit has examined Git from four angles: the personal and technical frustration that led to its creation, the legal framework that governs its use, its footprint on a Linux system, and its position in the broader FOSS ecosystem. What emerges is not just a picture of a version control tool, but a case study in why open source works.

Git succeeded not because it was marketed or funded by a corporation, but because one person had a real problem, built a solution in ten days, and made it available to everyone. The GPL v2 license ensured no company could reclaim it. The distributed model ensured no single failure point could stop it. And the quality of the tool meant adoption happened organically — today it is used by virtually every software developer on the planet.

The five shell scripts written alongside this report are a small demonstration of a larger point: Linux, Bash, and Git are not separate tools but parts of a coherent philosophy. They were built openly, documented thoroughly, and designed to be understood and modified. Learning to use them is not just acquiring technical skill — it is participating in a tradition of open knowledge that has shaped the modern world.

Every line of code I write in my career will run on foundations that others built freely and shared. Understanding why they made that choice — and what it demands of me in return — is not optional. It is part of being a professional.

---

7. References

- Torvalds, L. (2005). Initial Git release announcement. Linux Kernel Mailing List.
- Free Software Foundation. (2023). The Free Software Definition.gnu.org/philosophy/free-sw.html
- Open Source Initiative. (2023). The Open Source Definition. opensource.org/osd
- Raymond, E. S. (1999). The Cathedral and the Bazaar.catb.org/~esr/writings/cathedral-bazaar
- Git Project. (2024). Git Reference Manual. git-scm.com/docs
- Chacon, S. & Straub, B. (2014). Pro Git (2nd ed.). Apress. git-scm.com/book
- SPDX License List. (2024). GPL-2.0-only. spdx.org/licenses/GPL-2.0-only.html
- Shotts, W. (2019). The Linux Command Line (2nd ed.). linuxcommand.org
- Hamano, J. C. (2024). Git Mailing List Archive. lore.kernel.org/git/